An Efficient Semantic Query Optimization Algorithm

Hwee Hwa Pang[1], HongJun Lu[2], Beng Chin Ooi[1]
[1]Institute of Systems Science
[2]Department of Information Systems and Computer Science
National University of Singapore
Kent Ridge, Singapore 0511

Abstract

Semantic query optimization uses problem-specific knowledge, represented as semantic constraints, to answer queries efficiently. Although the potential cost savings from semantic query optimization have been amply demonstrated, so far few cheap and effective search strategy for an optimal set of query transformations have been reported. This paper is based on our experience in designing and implementing a prototype semantic query optimizer for an object-oriented database system. Our approach tentatively applies all the possible transformations and delays the choice of beneficial transformations till the end. This approach makes the order of transformations immaterial and makes it possible to have an algorithm for query transformations that is of polynomial complexity. Preliminary experiments on the prototype show that the optimizer performs well for large databases.

1. Introduction

Conventional query optimization methods attempt to determine a most efficient sequence of operations for the physical database system to perform, based on the syntax of a particular query and the access methods available. On the other hand, semantic optimization uses available semantic knowledge to transform a query into a new query which produces the same answer as the original query in *any* database state, and requires less system resources to execute. This optimization technique was first proposed by King [Kin81] and by Hammer and Zdonik [HaZ80]. The semantic knowledge required to perform the transformation is represented by a set of semantic constraints, which are also used to ensure the semantic validity of the database.

King's QUIST [Kin81] draws upon the knowledge of database structures and access methods to control the use of semantics for transformations. The approach proposed by Chakravarthy et al [CFM84] termed Semantic Compilation compiles the semantics (integrity constraints) together with the general laws of the system. Shenoy et al [ShO87, ShO89] used a graph theoretic approach to identify redundant join clauses and redundant predicates in a user query. Redundancies are eliminated and as many predicates on indexed attributes as possible are introduced. In [SSD88], Shekhar et al presented a first-order best first search algorithm to guide the query transformation process. Two criteria to terminate the search process were also discussed. All the above work use only knowledge about the problem domain expressed in the form of integrity constraints. As the set of integrity constraints is usually fixed, they do not change to reflect changes in database usage patterns. To overcome this limitation, Siegel [Sie88] extended the notion of semantic optimization to include system-derived rules that reflect the current database state. Besides facts that are always true of the database, the current database state also contains description of the current database status and hence captures more information. In his extension, a semantically equivalent query produces the same answer as the original query in the *current* database state. Yu and Sun [YuS89] also identified situations from previously processed queries where knowledge in the form of dynamic constraints can be automatically deduced.

The success of semantic query optimization depends largely on the development of an efficient algorithm for choosing among the many transformations made possible by the addition of any combination of semantic constraints to the query [JaK84]. In this paper we report our work which uses an efficient algorithm to implement existing semantic transformation rules. With our approach, all possible transformations are *tentatively* applied to the query. The task of choosing the beneficial transformations is delayed until all the possible transformations have been considered. Using this approach, transformations that have been applied do not preclude future transformations and the order of transformations is immaterial. Hence the algorithm is of polynomial complexity.

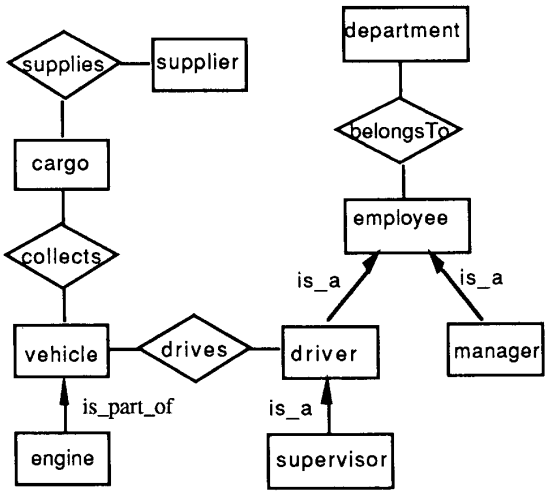
Another issue addressed in our design is the grouping of semantic constraints to reduce the overhead of retrieving constraints and checking whether each constraint is relevant

CH2968-6/0000/0326$01.00 © 1991 IEEE

to the current query. Constraints are grouped according to the object classes they reference. When optimizing a query, only selected groups of constraints need to be considered. At the same time, constraints are also classified according to the number of object classes each references. This classification will be exploited during runtime to help determine which predicates to retain and which to discard.

The remainder of this paper is organized as follows. Section 2 gives some background information about semantic query optimization and the issues to be addressed in this paper. Section 3 describes in detail our algorithm, which is based on a classification scheme for predicates in queries and semantic constraints. Section 4 analyses our approach and presents some performance data. Our conclusion is given in Section 5.

2. Background and Problem Definition



supplier(name, address, *supplies*)
cargo(code, desc, quantity, *supplies*, *collects*)
vehicle(vehicle#, desc, class, *engComp*, *collects*, *drives*)
engine(engine#, capacity, *engComp*)
employee(name, clearance, rank, *belongsTo*)
manager(name, clearance, rank, *belongsTo*)
driver(name, clearance, rank, *belongsTo*, license#, licenseClass, licenseDate, *drives*)
supervisor(name, clearance, rank, *belongsTo*, license#, licenseClass, licenseDate, *drives*)
department(name, securityClass, *belongsTo*)
Note. Attributes in *italic* are pointers used to implement relationships between object classes.

Figure 2.1: An Example Database

The work reported in this paper is part of a larger project to develop an object-oriented database system [LNO90]. Hence only those semantic transformation rules proposed by King [Kin81] that are applicable in the object-oriented context are considered. They are *restriction elimination*, *index and restriction introduction*, and *class elimination*. *Class introduction*, the equivalence of *relation introduction*, is not considered at the moment because, to decide when to introduce new classes and when to stop, the optimizer needs to estimate the costs of intermediate queries. This complicates the transformation process and makes the cost prohibitive. The example in Figure 2.3 illustrates the different transformation rules. This example uses the database in Figure 2.1 and the semantic constraints in Figure 2.2. The following format is used to represent queries.

(SELECT {projectList} {joinPredicateList}
{selectivePredicateList}
{relationshipList} {classList})

The different parts of the query describe the attributes required, the join predicates and selective predicates on object classes, the relationships between the classes involved, and the object classes to be accessed. Though there are some redundancies, this representation is nevertheless chosen to improve the clarity of our illustrations.
The transformation rules are briefly described below.

- *Restriction Elimination*: Some predicates are logical consequences of other predicates in a query. Such predicates might be removed to reduce the CPU time required to evaluate the query, since their removal or inclusion does not affect the results of the query. An example is transformation #2 in Figure 2.3.
- *Index and Restriction Introduction*: These transformations introduce additional predicates to the query in the hope of reducing the number of instances that need to be retrieved from an object class (and hence the size of intermediate results), or enabling other profitable transformations to be applied subsequently. They might also introduce predicates on indexed attributes. Transformation #1 in Figure 2.3 is one such example.
- *Class Elimination*: An object class with no selected attributes and non-optional predicates, and linked to just one object class, becomes dangling in the query and can be dropped [Kin81]. For example, transformation #3 in Figure 2.3.

In this paper only semantic constraints in the form of Horn clauses are considered, though our approach can easily be modified to handle other forms. Note also that, although only semantic constraints are used in the examples, rules

that reflect the current database state, such as those proposed by Siegel [Sie88], can easily be accommodated.

From the example in Figure 2.3, we observe some general issues in semantic query optimization.

1. Refrigerated trucks can only be used to carry frozen food.

c_1: cargo(_, desc, _, _, *collects*), vehicle(_, "refrigerated truck", _, _, *collects*, _) → equal(desc, "frozen food")

2. We get frozen food only from the Singapore Food Industries (SFI).

c_2: supplier(name, _, *supplies*), cargo(_, "frozen food", _, *supplies*, _) → equal(name, "SFI")

3. A driver can only drive vehicles whose classification is not higher than his license classification.

c_3: driver(_, _, _, _, _, licenseClass, _, *drives*), vehicle(_, _, class, _, _, *drives*)
→ greaterThanOrEqualTo(licenseClass, class)

4. Only research staff members can be appointed as managers.

c_4: manager(_, _, rank, _)
→ equal(rank, "research staff member")

5. Only employees whose security clearance is top secret can belong to the development department, which handles classified projects.

c_5: employee(_, clearance, _, *belongsTo*),
department("development", _, *belongsTo*)
→ equal(clearance, "top secret")

Figure 2.2: Semantic Constraints

- It is often expensive to determine whether a semantic transformation is beneficial. A related problem is the difficulty of deciding which predicates to introduce or eliminate.
- Depending on implementation, the order of transformations might be important because eliminating a predicate in the early stages may prevent the introduction of other predicates, hence the number of semantically equivalent queries is an exponential function of the number of possible transformations.
- The overhead of retrieving constraints and checking whether each constraint is relevant to the current query can be prohibitive, especially when there are many constraints.
- The costs of physically transforming queries are likely to be high.
- Special effort needs to be taken to prevent the introduction of predicates which were previously eliminated and vice versa, and to ensure termination.

Sample Query: List the vehicle# of refrigerated trucks that we sent to SFI to collect cargoes, and the description and quantity of the cargoes to be collected.

(SELECT {vehicle.vehicle#, cargo.desc, cargo.quantity} {}
{vehicle.desc = "refrigerated truck", supplier.name = "SFI"}
{collects, supplies}
{supplier, cargo, vehicle})

Transformation #1: Do restriction introduction using c_1.

(SELECT {vehicle.vehicle#, cargo.desc="frozen food", cargo.quantity} {}
{vehicle.desc = "refrigerated truck", cargo.desc = "frozen food",supplier.name = "SFI"}
{collects, supplies}
{supplier, cargo, vehicle})

Transformation #2: Do restriction elimination using c_2.

(SELECT {vehicle.vehicle#, cargo.desc="frozen food", cargo.quantity} {}
{vehicle.desc = "refrigerated truck", cargo.desc = "frozen food"}
{collects, supplies}
{supplier, cargo, vehicle})

Transformation #3: Do class elimination.

(SELECT {vehicle.vehicle#, cargo.desc="frozen food", cargo.quantity} {}
{vehicle.desc = "refrigerated truck", cargo.desc = "frozen food"}
{collects}
{cargo, vehicle})

Figure 2.3: Example of Semantic Query Optimization

Our semantic transformation algorithm is designed to address these problems. This algorithm is based on a classification scheme for semantic constraints and the predicates in queries. The quintessence of the algorithm is to avoid physically modifying queries during transformation, but to re-classify the predicates using existing classifications of the predicates and relevant semantic constraints. Finally the beneficial transformations are selected and the

transformed queries formulated according to the predicate classifications.

3. Query Transformation Algorithm

We first describe how we organize the semantic constraints to facilitate efficient retrieval, since this significantly affects the performance of our algorithm. In our system, the transitive closures of the constraints are materialized during precompilation. This involves computing the closure of existing predicates using domain knowledge, eg. if (A = a) $\rightarrow$ (B > 20) and (B > 10) $\rightarrow$ (C = c) then deduce (A = a) $\rightarrow$ (C = c). The algorithm is similar to the one proposed in [YuS89]. The advantage is it is no longer necessary to dynamically compute the transitive closures for each query. This simplifies the optimization process. The possible disadvantages of storing the transitive closures are the necessity to re-compute the closures when constraints are updated, additional storage requirements and the problem of dealing with more constraints during runtime. The first problem is not serious since semantic constraints are not usually changed frequently. The second problem can easily be overcome by extracting all the predicates into a separate structure, and modifying the constraints to contain only pointers to relevant predicates in the structure. This way the additional storage required is minimal. The last problem of having to deal with more constraints is also not a major issue, since we have a grouping scheme (which will be discussed shortly) to effectively handle large number of constraints. We believe that the runtime savings derived from storing the closures more than justify the overhead.

In designing a grouping scheme for the constraints, we observe that all the semantic constraints that can be used to effect restriction elimination, index/restriction introduction, and class elimination have one common property: each such constraint refers only to object classes that appear in the query. (Recall that we do not intend to introduce new object classes during semantic optimization.) A semantic constraint c_i is said to be *relevant* to a query q if and only if all the object classes c_i references also appear in q. Note that this is true only because the transitive closures are materialized. We exploit this property to design a simple grouping scheme that reduces the number of constraints that need to be considered for each query. In this scheme, constraints are grouped according to the object classes they reference. A constraint is arbitrarily assigned to a group g_k, which is attached to object class o_k and o_k is one of the object classes referenced in the constraint. To optimize a query, only those groups of constraints attached to object classes that appear in the query need to be considered. Thus some irrelevant constraints are eliminated right away.

To retrieve relevant constraints for a query, all constraints attached to object classes referenced in the query are retrieved. Thus all the relevant constraints will always be retrieved. Suppose constraint c_i is relevant to a query q, and c_i is assigned to group g_k. Object class o_k appears in c_i, so it must appear in q by the definition of relevance of constraints. Our scheme will retrieve g_k (and hence c_i). Thus the grouping scheme is correct, though not necessarily optimal.

With this simple grouping scheme, some irrelevant constraints will be retrieved as well. To reduce the number of irrelevant constraints retrieved, the statistics on access frequency of each object class is maintained , and semantic constraints are assigned to the group attached to the less frequently accessed classes that appear in the constraint:

$g_k \leftarrow c_i$, where o_k is the least frequently accessed among the object class in c_i.

With this enhancement, constraints that refer to classes that are seldom accessed would normally not be considered since typical queries do not involve these classes. With this improvement, normally most of the irrelevant constraints would not be considered. However, a drawback with this enhancement is the grouping has to be updated as database access pattern changes. An alternative would be to distribute constraints as evenly as possible among the groups.

Our algorithm comprises four components as shown in Figure 3.1. After initializing the necessary data structures, the transformation queue is updated to contain all the transformations that can be performed. The first transformation in the queue is then carried out and the queue updated again. This process continues until the queue is empty immediately after getting updated. The final step formulates the transformed query.

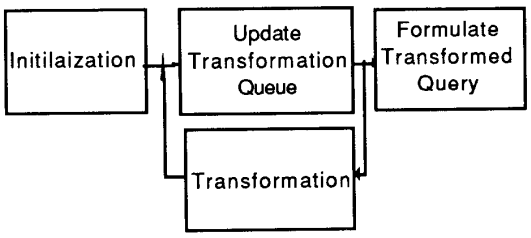


Figure 3.1: Overview of Semantic Query Optimization

3.1 Initialization

This step initializes the data structures used to support query transformations.

In our algorithm, predicates in a query are classified as either *imperative*, *optional*, or *redundant*. An *imperative* predicate is one whose removal will affect the final results. An *optional* predicate is one that, though its inclusion or exclusion does not affect the final results, might affect the execution efficiency of the query by enabling us to make use of indices, or by cutting down the number of instances returned from an object class and thereby reducing the size of intermediate results. Whether an *optional* predicate should eventually be retained in the query depends on the estimated cost savings it helps bring about and the cost of evaluating it. A *redundant* predicate affects neither the final results nor execution efficiency. Each predicate is assigned a tag that shows its classification.

Definition: The tag of predicate p_j is $t_p(p_j)$, where $t_p(p_j) = a, a \in$ {imperative, optional, redundant}.

Upon receiving a query, the groups of semantic constraints assigned to those object classes referenced in the query are fetched into memory. Each constraint is checked to determine if all the object classes it refers to appear in the query. If so, the constraint is relevant. ***C***, the set of relevant constraints, ***P***, the set of predicates that appear either in the query or the relevant constraints, and the transformation table ***T*** are constructed. We begin by making all the predicates in the query *imperative*. This is because, unless proven otherwise, we have to assume that all the predicates contribute to the results of the query.

- $\boldsymbol{C} = \{c_1, .., c_m\} = \{c \mid c$ is a relevant semantic constraint$\}$.
- $\boldsymbol{P} = \{p_1, .., p_n\} = \{p \mid p$ is a predicate in semantic constraint c, c in $\boldsymbol{C}\} \cup \{p \mid p$ is a predicate in the query$\}$.
- $t(c_i, p_j)$ is a cell in row i and column j of $\boldsymbol{T}$, and $t(c_i, p_j) \in$ {AbsentAntecedent, PresentAntecedent, AbsentConsequent, Imperative, Optional, Redundant, _}.

The terms are all descriptions of predicates and their meanings are obvious. For example, $t(c_i, p_j)$ = AbsentAntecedent means predicate p_j appears in the antecedent of constraint c_i but not in the query, and $t(c_i, p_j)$ = _ means predicate p_j does not appear in constraint c_i.

Algorithm: *Initialization*

```
/* Build transformation table T */
for each object class o_k in the query
    retrieve into C relevant semantic constraints from g_k;
    collect all the predicates into P
endfor;

/* Initialize transformation table T */
for each c_i in C
    for each p_j in P
        if p_j is a consequent predicate in c_i then
            if p_j appears in the query then
                t(c_i, p_j) := Imperative;
            else /* p_j does not appear in the query */
                t(c_i, p_j) := AbsentConsequent
            endif;
        else
            if p_j is an antecedent predicate in c_i then
                if p_j appears in the query then
                    t(c_i, p_j) := PresentAntecedent;
                else /* p_j does not appear in the query */
                    t(c_i, p_j) := AbsentAntecedent;
                endif
            else /* p_j does not appear in c_i */
                t(c_i, p_j) := _
            endif
        endif
    endfor
endfor;
```

3.2 Update Transformation Queue

This component identifies all the transformations that can be performed and places them in a queue.

We distinguish between two classes of semantic constraints - *intra-class* and *inter-class*. Each *intra-class* constraint makes reference to attributes of only one object class, eg. c_4 in Figure 2.2. All constraints that refer to attributes of more than one object class are said to be *inter-class*, in the sense that they relate attributes across object classes. All the semantic constraints in Figure 2.2 except c_4 are *inter-class* constraints. During precompilation all the semantic constraints are thus classified and the classification is stored in the tags associated with them. These tags are used to help classify predicates during query transformation.

Definition: The tag of semantic constraint c_i is $t_c(c_i)$, where $t_c(c_i) = a, a \in$ {intra, inter}.

A semantic constraint can only be "fired" to effect transformations if all its antecedent predicates are present. A transformation queue ***Q*** (initially empty) is used to hold the list of semantic constraints that can be "fired". During the transformation process, successive constraints are taken

from Q to effect transformations, and T is updated accordingly. As a result of these transformations, further transformations might be made possible and the corresponding constraints are added to Q. The transformation process terminates when Q is empty. The algorithm is outlined as follows.

Algorithm: *Update Transformation Queue Q*

```
for each semantic constraint c_i in C
  let its consequent predicate be p_j;
  case t(c_i, p_j) of
    Redundant :/* t(c_i, p_j) cannot be lowered further */
          remove c_i from C;
    Optional :   /* consequent predicate is optional */
        case t_c(c_i) of
            intra :  /* c_i might be used to lower t(c_i, p_j) */
              if t(c_i, p_k) = PresentAntecedent for all
                    antecedent predicates p_k of c_i then
                insert c_i into Q;
              endif
            inter :  /* c_i cannot be used to lower t(c_i, p_j) */
                remove c_i from C
        endcase;
    Imperative :   /* c_i can be used to lower t(c_i, p_j) */
    AbsentConsequent :
        /* c_i can be used to introduce p_j int the query */
        if t(c_i, p_k) = PresentAntecedent for all
                antecedent predicates p_k of c_i
         then insert c_i into Q
         endif
  endcase
endfor;
```

3.3 Transformation

During transformation, constraints are taken from Q and the corresponding transformations are carried out. Each transformation changes the tags of the affected predicates, instead of physically modifying the query. The transformation algorithm is based on Tables 3.1 and 3.2, which indicate how the tags should be changed when a transformation is applied. The tables are constructed according to the following reasoning: a predicate that is implied by other predicates in a query cannot affect the final results if it is removed or introduced. Consider the case where an *intra-class* constraint (i.e. all the antecedent and consequent clauses of the constraint refer to attributes in the same object class) is used to effect the transformation. If the consequent predicate is on an indexed attribute (such a predicate is called an indexed predicate), it inclusion might help in reducing the number of object instances that need to be retrieved, hence it is *optional*. If the consequent predicate is not an indexed predicate, its presence or absence does not affect execution efficiency since the instances that will be returned from this object class are already determined by the antecedent predicates. Hence the consequent predicate is *redundant*. Now consider the case where an *inter-class* constraint (i.e. some predicates in the antecedent or consequent refer to attributes of different object classes) is used. The presence or absence of the consequent predicate might affect execution efficiency because it might get evaluated before the antecedent predicates and thus reduce the size of intermediate results. Hence the consequent predicate is *optional*.

A restriction elimination using a semantic constraint lowers the tag of the constraint's consequent predicate, depending on the classification of the constraint. The new tag is determined from Table 3.1, where X stands for "don't care". For example, transformation #2 in Figure 2.3 would change the *imperative* tag of *supplier.name = "SFI"* to *optional*, since c_2 is an *inter-class* semantic constraint.

Index introduction and restriction introduction bring in new predicates. The tags assigned to the new predicates again depend on the semantic constraints used to effect the transformations and are determined from Table 3.2, where X stands for "don't care". Transformation #1 in Figure 2.3, for example, introduces the predicate *cargo.desc = "frozen food"* with a *optional* tag as c_1 is an *inter-class* semantic constraint.

Table 3.1: Changes to tags by Restriction Elimination

Semantic Constraint	Consequent Predicate	Tag		
		imperative	optional	redundant
Intra-Class	not indexed	redundant	optional	not applicable
	indexed	optional	not applicable	not applicable
Inter-Class	X	optional	not applicable	not applicable

Table 3.2: Tags assigned through Index/Restriction Introduction

Semantic Constraint	Consequent Pred.	Tag
Intra-Class	not indexed	***redundant***
	indexed	***optional***
Inter-Class	X	***optional***

Algorithm: *Transformation*

```
while Q is not empty
   remove the first semantic constraint, c_i, from Q;
   newTag := null;
   suppose p_j is the consequent predicate of c_i;
case t_c(c_i) of
   intra : if t(c_i, p_j) = Imperative or Optional
                  /* lower t(c_i, p_j) */
           or t(c_i, p_j) = AbsentConsequent then
           /* introduce p_j */
              newTag := Redundant
           /* else some c_k ahead of c_i in Q has
           already lowered t(c_i, p_j) - ignore c_i then */
         endif;
   inter : if t(c_i, p_j) = Imperative   /* lower t(c_i, p_j) */
         or t(c_i, p_j) = AbsentConsequent then   /*
introduce p_j */
              newTag := Optional
         /* else some c_k ahead of c_i in Q has already
lowered t(c_i, p_j) - ignore c_i then */
         endif
   endcase;
   if newTag ≠ null then
      t(c_i, p_j) := newTag;/* lower t(c_i, p_j) */
      for each c_k in C    /* update column c_k in T
*/
         case t(c_k, p_j) of
            AbsentAntecedent :
               t(c_k, p_j) := PresentAntecedent;
            Imperative, Optional, Redundant:
               t(c_k, p_j) := newTag
         endcase
      endfor
   endif
endwhile;
```

3.4 Query Formulation

After the transformation process, the tag $t_p(p_j)$ of each predicate p_j is determined from T. At this stage, it might be desirable to apply the class elimination rule. Note that the absence of *imperative* predicates on its attributes is a necessary, though not sufficient, condition for an object class to be eliminated. The profitability of removing a class from the query can be estimated using the cost model in the conventional query optimizer. The decision to retain or discard a predicate depends on its tag and is shown in Table 3.3. This table is based on the definitions of *imperative*, *optional* and *redundant*. The *optional* predicates to retain are chosen based on the estimated profitability of each such predicate. Those *optional* predicates that are not found to be profitable would be re-classified as *redundant*. The final query contains only the *imperative* and *optional* predicates.

Table 3.3: Retain or discard predicates

TAG	imperative	optional	redundant
ACTION	retain	cost-benefit analysis	discard

Algorithm: *Query Formulation*

```
for each predicate p_j
   if t(c_i, p_j) = Optional or Redundant for some c_i
then
      t_p(p_j) := t(c_i, p_j);
   else
      t_p(p_j) := Imperative
   endif
endfor;
apply class elimination rule if desirable;
for each optional predicate p_j, i.e. t_p(p_j) = Optional
   if not profitable(p_j) then
      /* discard non-profitable optional predicates */
      t_p(p_j) := Redundant
   endif
endfor;
formulate query using only imperative and optional
predicates;
```

Function profitable(p_j) in the above algorithm determines whether it is profitable to retain p_j in the final query. This is done by estimating the possible cost savings and overhead of retaining p_j, using a cost model and conventional query

optimization techniques and will not be discussed further here.

3.5 An Example

We now illustrates how the semantic query optimization shown in Figure 2.3 would be done using our algorithm.

Step 1: Initialization

$C = \{c_1, c_2\}$ where c_1 and c_2 are semantic constraints in Figure 2.2.

$P = \{p_1, p_2, p_3\}$ where p_1 = (vehicle.desc = "refrigerated truck"), p_2 = (supplier.name = "SFI"), and p_3 = (cargo.desc = "frozen food").

$$T = \begin{pmatrix} \text{PresentAntecedent} & \_ & \text{AbsentConsequent} \\ \_ & \text{Imperative} & \text{AbsentAntecedent} \end{pmatrix}$$

$Q = \{c_1\}$

Step 2: Transformations

Transformation #1: Restriction introduction using c_1.

$$T = \begin{pmatrix} \text{PresentAntecedent} & \_ & \text{Optional} \\ \_ & \text{Imperative} & \text{PresentAntecedent} \end{pmatrix}$$

$Q = \{c_2\}$

Transformation #2: Restriction elimination using c_2.

$$T = \begin{pmatrix} \text{PresentAntecedent} & \_ & \text{Optional} \\ \_ & \text{Optional} & \text{PresentAntecedent} \end{pmatrix}$$

$Q = \{\ \}$

Step 3: Formulate transformed query

p_1 is *imperative*, p_2 and p_3 are *optional*. By eliminating the supplier class, p_2 is dropped. Hence the transformed query is

(SELECT {vehicle.vehicle#, cargo.desc="frozen food", cargo.quantity} {}
{vehicle.desc = "refrigerated truck", cargo.desc = "frozen food"}
{collects}
{cargo, vehicle})

4. Discussion and Some Empirical Results

If some transformations are deemed more desirable than others, eg. index introduction is likely to be more profitable than predicate elimination, and predicate elimination is preferred over predicate introduction, priorities can be assigned to different transformation rules and Q becomes a priority queue. This enhancement is very useful when it is necessary to assign a budget and limit the number of transformations, in which case we would like to perform those transformations that are more likely to be profitable first.

With a reasonably accurate cost model in the conventional optimizer, our approach will always apply all the beneficial transformations. A straight-forward approach to do semantic optimization is to evaluate the profitability of each transformation, and if deemed profitable, immediately apply it to the query. This way, some transformations might preclude other transformations (eg. eliminating an antecedent predicate of a semantic constraint means it cannot be used to introduce its consequent predicate) and hence the order of transformations is important. On the other hand, our approach can be visualized as tentative applications of all the possible transformations. Since a transformation does not have immediate effect on the query, it does not preclude other transformations. So the order of transformations is immaterial. Only after all possible transformations have been considered do we select those that are beneficial and make their effects permanent. Hence, all the transformations that are eventually selected are all deemed profitable, and would correspond to the selected transformations using the straight-forward approach, i.e. the outcome using our approach is at least as good as that using the straight-forward approach.

In analyzing our algorithm, the retrieval of relevant semantic constraints in the initialization step and the cost-benefit analyses in the query formulation step are not considered. This is a fair assumption because our main objective is to provide an efficient algorithm for query transformation, and we are only interested in the performance of this step. In any case, the costs that are not take into account will also be incurred by all the other query transformation algorithms. Since the order of transformations is immaterial, our query transformation step is bounded by $o(mn)$, where m is the number of distinct predicates in the query and the relevant constraints, and n is the number of relevant semantic constraints. Another advantage of our approach is it is only necessary to test the profitability of a subset of transformations, as some profitable transformations (those that reduces the classifications of predicates to *redundant*) can straight away be identified.

Our prototype semantic query optimizer was implemented on a SUN-3/160 workstation. As different parts of our object-oriented database system (OODB) are currently being developed, the effectiveness of our optimizer was evaluated as follows. A sample database whose schema is shown in Figure 2.1 was built. All possible paths in this schema were identified, where a path consists of a series of interconnecting object classes and relationships, and no object class or relationship appears more than once. A query was formulated for each such path and thus a set of queries was generated. From this set of queries, 40 test queries were randomly chosen and sent to the optimizer. After optimization, each pair of original and optimized query was sent to a DBMS to be executed. We measured the query transformation times and cost savings, the two main factors that determine the effectiveness of the optimizer. A relational DBMS was used to simulate the cost ratios of the optimized and original queries. Although relational DBMS and OODB might use different access mechanisms, we believe an improvement in execution efficiency in the relational context would most likely indicate a similar improvement in OODB context.

In our experiment, each object class had an average of 3 semantic constraints attached to it. The optimizer was tested with four database instances, each of increasing size. Some statistics of the database instances are given in Table 4.1.

Table 4.1 Database Sizes

	DB1	DB2	DB3	DB4
# object class	5	5	5	5
avg. class cardinality	52	104	208	208
# relationships	6	66	6	6
avg. relationship cardinality	77	154	308	616

Figure 4.1 summarizes the query transformation times for the 40 randomly generated queries. The results show that query transformation time is clearly proportional to both the number of object classes in the query and, to a lesser extent, the number of relevant constraints. For all the queries tested, the total query transformation time (including retrieval of semantic constraints) is less than a second for each query. Subtracting the I/O retrieval time, the maximum time spent on actual transformation is less than 0.4 seconds. We feel that such performance is reasonably good.

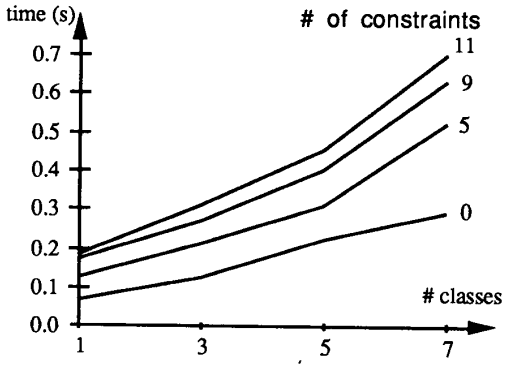


Figure 4.1: Query Transformation Time

Table 4.2 shows the cost saving ratio of the cost of optimized query (including query transformation time) and the cost of original query for each of the 40 pairs of queries and each database instance. The table shows that for DB1, the smallest database used, performance worsened for 40% of the queries, and the extra overheads were limited to about 10%. Only 34% of the queries executed faster after optimization, out of which 20% were improved significantly. This results were expected because when the database is small, retrieval times of the original queries are low (in our experiment most of them took 1 to 2 seconds) and so unless the output can be obtained without going to the database, the overhead of optimization usually more than offsets the little savings that can be derived. As the database is large, optimization is much more effective. For DB4, the largest database used, 67% of the queries executed faster after optimization. Furthermore, some queries (27%) which

Table 4.2: Ratio of Optimized Cost and Original Cost

	0%	10%	20%	30%	40%	50%	60%	70%	80%	90%	100%	110%
DB1	_	20	_	_	_	_	_	_	7	7	26	40
DB2	_	20	_	_	_	_	_	_	27	7	13	33
DB3	13	13	7	_	_	_	7	7	13	_	40	_
DB4	27	13	_	_	7	_	_	7	13	_	33	_

originally took hours to execute or were aborted because the system ran out of resources were able to be executed much faster.

We conclude, as one may expect, that it is probably not worth doing semantic query optimization when the database is small or when the query execution cost is expected to be low, i.e. the semantic query optimizer should be disabled. However when the database is large or when the query execution cost is expected to be high, the optimizer becomes very useful.

5. Conclusion

The most important contribution of this paper is the proposal of an efficient semantic query optimization algorithm. With our approach, all possible transformations are *tentatively* applied to the query. Instead of physically modifying the query, the transformation process classifies the predicates into *imperative*, *optional*, or *redundant*. At the end of the transformation process, all the *imperative* predicates are retained while the *redundant* predicates are eliminated. *Optional* predicates are retained or discarded based on the estimated cost/benefit of retaining them. This effectively means the task of choosing the beneficial transformations is delayed until all the possible transformations have been considered. Using this approach, previous transformations do not preclude other transformations and the order of transformations is immaterial. Hence the algorithm is of polynomial complexity. Another advantage of this approach is there is no need to check the profitabilities of all the transformations, eg. those transformations that re-classify predicates to *redundant* should always be carried out. Another issue addressed in this paper is the grouping of semantic constraints to reduce the overhead of retrieving constraints and checking whether each constraint is relevant to the current query. Constraints are grouped according to the object classes they reference. When optimizing a query, only selected groups of constraints need to be considered. At the same time, constraints are also classified as *intra-* or *inter-class*, depending on the number of object classes each references. This classification is exploited during runtime to help determine the predicates to retain or discard. A prototype semantic query optimizer based on our algorithm has been built and preliminary experiments show that the optimizer performs well for large databases.

References

[CFM84] U.S. Chakravarthy, D.H. Fishman, J. Minker, "*Semantic Query Optimization in Expert Systems and Database Systems*", Proc. of 1st Int. Workshop on Expert Database Systems, Oct 1984.

[CMG86] U.S. Chakravarthy, J. Minker, J. Grant, "*Semantic Query Optimization: Additional Constraints and Control Strategies*", Proc. of the 1st Int. Conf. on Expert Database Systems, Apr 1986.

[HaZ80] M.M. Hammer, S.B. Zdonik, "*Knowledge Based Query Processing*", Proc. of the 6th Int. Conf. on Very Large Data Bases, Sep 1980.

[JaK84] M. Jarke, J. Koch, "*Query Optimization in Database Systems*", ACM Computing Surveys, Vol. 16, No. 2, June 1984.

[Kin81] J.J. King, "*QUIST: A System for Semantic Query Optimization in Relational Databases*", Proc. of the 7th Int. Conf. on Very Large Data Bases, Sep 1981.

[LNO90] H. Lu, A. Ngu, B. Ooi, H. Pang, R. Price, L. Wong, J. Lim, "*On the Design of a Multimedia Object-Oriented Database System*", submitted for publication, 1990.

[POL90] H. Pang, B. Ooi, H. Lu, "*An Integrated Query Optimizer for OODB*", ISS Technical Report TR90-18-0, 1990.

[ShO87] S.T. Shenoy, Z.M. Ozsoyoglu, "*A System for Semantic Query Optimization*", Proc. of the ACM SIGMOD 1987 Annual Conference.

[ShO89] S.T. Shenoy, Z.M. Ozsoyoglu, "*Design and Implementation of a Semantic Query Optimizer*", IEEE Transactions on Knowledge and Data Engineering, Vol. 1, No. 3, Sep 1989.

[Sie88] M.D. Siegel, "Automatic Rule Derivation for Semantic Query Optimization", Proc. of the 2nd Int. Conf. on Expert Database Systems, Apr 1988.

[SSD88] S. Shekhar, J. Srivastava, S. Dutta, "*A Formal Model of Trade-off between Optimization and Execution Costs in Semantic Query Optimization*", Proc. of the 14th Int. Conf. on Very Large Data Bases, 1988.

[YuS89] C.T. Yu, W. Sun, "*Automatic Knowledge Acquisition and Maintenance for Semantic Query Optimization*", IEEE Transactions on Knowledge and Data Engineering, Vol. 1, No. 3, Sep 1989.